



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Universitats i Ocupació

ceedcv

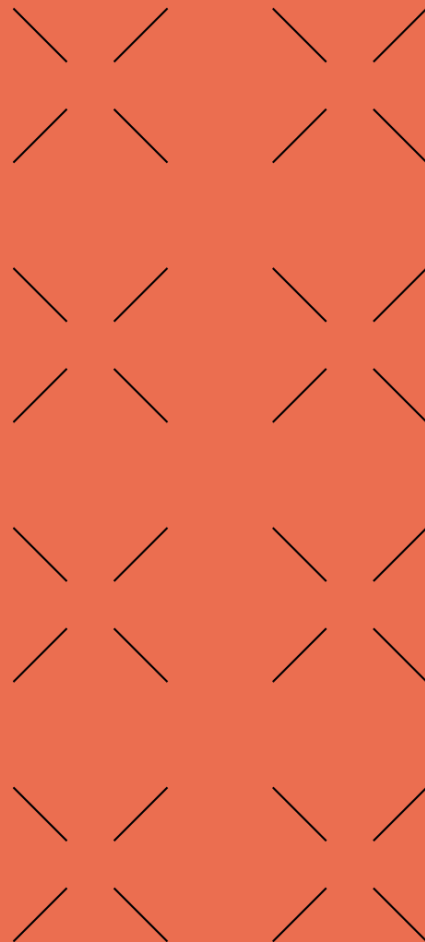
CENTRE ESPECÍFIC
D'EDUCACIÓ A DISTÀNCIA DE
LA COMUNITAT VALENCIANA

Unitat 04 — Estructures de control i excepcions

Bloc 03

1r CFGS DAW · Programació

Sergi García Barea



| try, catch i finally

La idea en una frase

“

`try` protegix el codi perillós, `catch` atrapa l'excepció si apareix i `finally` s'executa **sempre**, ocorrega el que ocorrega.

En el punt 6 vas vore que una `InputMismatchException` mata el teu programa. Ara toca blindar-lo: el `try/catch/finally` és l'airbag del codi.

L'estructura completa

LectorBlindat.java

Java

```
try {  
    System.out.print("Quants anys? ");  
    int edat = sc.nextInt();  
    System.out.println("Fa " + edat + " anys.");  
} catch (InputMismatchException e) {  
    System.out.println("Això no és un nombre.");  
}  
  
System.out.println("El programa segueix viu. 🎉");
```

- 1 **try**: el codi perillós, sota vigilància
- 2 **catch**: què fer si apareix eixa excepció
- 3 **finally** (opcional): s'executa **SEMPRE**, amb o sense excepció, fins i tot amb un `return` al try. Ideal per a tancar Scanner i fitxers

Captura múltiple: l'ordre importa

MultiCatch.java

Java

```
try {  
    System.out.println(numeros[5] / 0);  
} catch (ArithmeticException e) {  
    System.out.println("Divisió entre zero.");  
} catch (ArrayIndexOutOfBoundsException e) {  
    System.out.println("Índex fora.");  
} catch (RuntimeException e) {  
    System.out.println("Alguna estranya.");  
}
```



Un `catch (Exception e)` al principi es menjaria les excepcions més específiques. Regla: **del més concret al més general**, com en els `else if`.



Des de Java 7, diversos tipus amb el mateix tractament caben en un sol `catch`:

```
catch (A | B e)
```

La variable e: el botí de l'error

Boti.java

Java

```
catch (Exception e) {  
    System.out.println(  
        "Missatge: " + e.getMessage());  
    e.printStackTrace();  
}
```

e

L'objecte excepció atrapat:

e.getMessage() dona el missatge;

e.printStackTrace() el rastre complet
per a depurar



Un catch **buit** és un pecat mortal: te tragues l'error i ni tan sols saps que ha passat. Com a mínim, imprimix un missatge.

Exemple guiat: el menú a prova de bombes

MenuBlindat.java

Java

```
do {
    System.out.println("1. Jugar  2. Eixir");
    System.out.print("Tria: ");
    try {
        opcio = sc.nextInt();
        valida = (opcio == 1 || opcio == 2);
    } catch (InputMismatchException e) {
        System.out.println("Això no és un nombre.");
        sc.next(); // descarta el text brossa
    }
} while (!valida);
```

 Fixa't en `sc.next()` dins del catch: sense ell, el text brossa seguiria al buffer i el següent `nextInt()` tornaria a fallar. Atrapar l'excepció i **netejar el buffer** són dos passos del mateix ball.

Mini-chequeig: try/catch



Respon en 30 segons:

1. Què fa el bloc finally i quan s'executa?
2. En quin ordre han d'anar els catch?
3. Quin perill té un catch buit?
4. Per què convé cridar `sc.next()` després d'una `InputMismatchException`?



Respostes:

- S'executa **sempre**, hi haja o no excepció; servix per a netejar recursos
- **Del més específic al més general**; si no, el general "se menja" els altres i no compila
- Te tragues l'error sense assabentar-te'n: fallada oculta. Mínim: imprimix un missatge
- Perquè el text brossa es queda al buffer i el següent `nextInt()` tornaria a fallar

throw: llança la pedra

La idea en una frase

“

`throw` llança una excepció quan **tu** decideixes que alguna cosa no ha de continuar, i creant la teua pròpia excepció pots posar-li nom al problema.

Fins ara Java llançava les excepcions per tu. Ara arriba el superpoder: decidir tu quan llançar-les, i inventar-te tipus d'error a la teua mesura.

throw en acció

CompteBancari.java

Java

```
double saldo = 10.0;
double retir = 500.0;
if (retir > saldo) {
    throw new ArithmeticException(
        "Saldo insuficient: " + saldo);
}
saldo -= retir;
```

 El missatge que li passes al constructor és el que veuràs en `e.getMessage()` . I compte: `throw` **llança** (al cos del mètode); `throws` **anuncia** en la signatura que el mètode pot llançar excepcions controlades.

Excepcions pròpies: el teu defecte a mida

SaldoInsuficientException.java

Java

```
public class SaldoInsuficientException
    extends RuntimeException {
    public SaldoInsuficientException(
        String missatge) {
        super(missatge);
    }
}
```

Caixer.java

Java

```
if (retir > saldo) {
    throw new SaldoInsuficientException(
        "Només tens " + saldo + "€.");
}
```

✓ **Hereta d'Exception** si vols **obligar** a capturar-la (checked). **De RuntimeException** si prefereixes que no l'obligue (unchecked): per a començar, és la còmoda.

Checked vs unchecked: la burocràcia



Checked (controlades)

- El compilador **t'obliga** a capturar-les (try/catch) o a declarar-les (`throws`)
- Herenen d'Exception, però no de RuntimeException
- Exemple: `IOException`



Unchecked (no controlades)

- No t'obliguen a res: si no les gestiones, exploten en execució
- Són RuntimeException i les seues filles
- Exemples: NPE, ArithmeticException...



Si una excepció és checked i no la gestiones, **no compila**. Si és unchecked, el compilador et deixa tranquil... i l'error explota en plena execució.

Exemple guiat: la màquina expendedora

MaquinaExpenedora.java

Java

```
try {  
    if (stock == 0) {  
        throw new ProducteEsgotatException(producte);  
    }  
    System.out.println("Ací tens el teu " + producte);  
} catch (ProducteEsgotatException e) {  
    System.out.println("Ho sentim: " + e.getMessage());  
}  
System.out.println("La màquina segueix funcionant. 🤖");
```

Ho sentim: El producte Refresc està esgotat.
La màquina segueix funcionant. 🤖

✅ El throw llança la teua excepció, el catch l'atrapa pel seu **nom propi** i el programa sobreviu: un error genèric es converteix en un missatge que fins i tot la teua cap entén.

Mini-chequeig: throw



Respon sense mirar:

1. Què fa la paraula clau throw?
2. Com es crea una excepció pròpia?
3. Quina és la diferència entre throw i throws?
4. Quina diferència hi ha entre checked i unchecked?



Respostes:

- **Llança** una excepció on la col·loques: `throw new MiExcepcion("...")`
- Heredant d'Exception o RuntimeException + constructor amb `super(missatge)`
- `throw` llança (cos del mètode); `throws` declara (signatura)
- **Checked:** obliga a capturar o declarar; **unchecked:** no obliga (RuntimeException)

Repàs: qui té la raó?

Fireside Chat: if-else vs switch

if-else diu:

- "Jo soc el clàssic: rangs, comparacions, mescles"
- "Tu només servix per a valors exactes..."

switch diu:

- "Amb mi poses la variable una vegada i cada cas en la seua línia"
- "I tu? Amb trenta else if, saps quin va abans que quin?"

✅ **La lliçó:** no hi ha guanyador. **switch** per a valors concrets (dia, menú, talla) i **if/else** if per a rangs i mescles. Triar bé és la mitat de l'examen.

Repàs: eres la JVM

Misteri.java

Java

```
int nota = 6;
if (nota ≥ 5) {
    System.out.println("Aprovat");
} else if (nota ≥ 7) {
    System.out.println("Notable");
} else {
    System.out.println("Suspés");
}
```

Què imprimix? a) Notable · b) Aprovat · c) Suspés



Resposta: b) Java avalua de dalt a baix i es queda amb la **primera** condició true. $6 \geq 5 \rightarrow$ "Aprovat", encara que no arribe a 7. Si vols que un 6 siga "Notable", reordena de més exigent a més permissiva.

Repàs: el joc de les decisions



Tria la resposta correcta:

1. `int n = 4; String r = n >= 5 ? "A" : "B";` → a) A · b) B
2. Quantes voltes fa `for (int i = 0; i < 3; i++)` ? → a) 3 · b) 4
3. switch sense break, la variable val 1 → a) només case 1 · b) case 1 i 2
4. Què dona `10 / 0` ? → a) ArithmeticException · b) un nombre enorme



Solucions:

- **b)** — 4 no és ≥ 5 : el ternari retorna "B"
- **a)** — i val 0, 1 i 2: tres voltes
- **b)** — sense break, fall-through al case 2
- **a)** — dividir entre zero llança ArithmeticException

CONRAD vs el món: "el bucle que no s'acaba"

- 1 Un while **sense res que canvie la condició** dins és un cotxe sense frens
- 2 `while (x > 0) { x = x + 1; }` : puja en comptes de baixar. "Com si volgueres buidar una piscina tirant-hi més aigua"
- 3 `if (x = 5)` amb UN igual: no és una condició, és una **assignació**!



Abans d'acusar l'ordinador de "congelar-se", mira el bucle: **alguna cosa modifica la condició cap a false?** El `continue` es salta l'actualització? Uses `=` o t'has quedat en `=` ? El 90% dels programes "penjats" s'arreglen ací.

Entrevista de treball

Les preguntes que et faran

1

Explica com si jo fóra la teua
àvia la diferència entre if i
switch

2

Quina és la diferència entre
break i continue?

3

L'usuari escriu text on
esperàvem un nombre i l'app es
cau. Com ho arregles?

4

Què és una
NullPointerException i com
l'evites?

5

Per a què servix el bloc finally?

6

Quan crearies una excepció
pròpia?

No hi ha preguntes tontes



Puc usar switch amb un double?

No: admet int, char, enum i String. Amb double, if/else if (els decimals no es comparen amb ==)



Per què veig while (true)?

Es trenca des de dins amb break: "per sempre fins que passe alguna cosa". Molt comú en jocs i servidors



El catch pot capturar tot?

catch (Exception e) atrapa totes les Exception (i RuntimeException). catch (Throwable e) és pescar amb dinamita

Resum de la unitat



if / switch

Semàfor: guanya el primer true ·

Carta: case + break + default



while / do / for

Comprova abans vs després ·

Comptador de sèrie ·

break/continue



Excepcions

try/catch/finally · throw i pròpies

· checked vs unchecked

